

MEDS Lab Module 4

Section 11: Finite State Machines

Task 14: Custom-Sequence Counter with T Flip-Flops

July 23, 2026

1. EDA Playground Link

The complete design and testbench code for the custom-sequence counter can be found at the following shareable link:

- <https://edaplayground.com/x/AStG>

2. Schematic Diagram & State Table

Below is the combined visual representation including the state transition diagram, the derived excitation state table, and the final gate-level circuit implementing the T flip-flops and next-state logic.

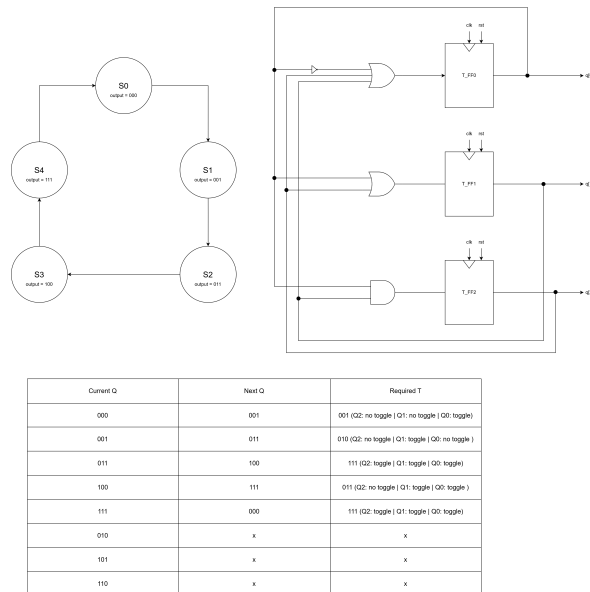


Figure 1: State diagram, excitation table, and gate-level circuit for the custom sequence.

3. Terminal Output Screenshot

The following screenshot displays the simulation log, confirming that the counter successfully traverses the sequence $0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 0$ continuously for multiple cycles.

```
Compiler version X-2025.06-SP1_Full164; Runtime version X-2025.06-SP1_Full164; Jul 23 08:15 2026
[PASS] q = 0
[PASS] q = 1
[PASS] q = 3
[PASS] q = 4
[PASS] q = 7
[PASS] q = 0
[PASS] q = 1
[PASS] q = 3
[PASS] q = 4
[PASS] q = 7
[PASS] q = 0
[PASS] q = 1
[PASS] q = 3
[PASS] q = 4
[PASS] q = 7
$finish called from file "testbench.sv", line 35.
$finish at simulation time      162
      V C S  S i m u l a t i o n  R e p o r t
Time: 162 ns
CPU Time:      0.470 seconds;      Data structure size:  0.0Mb
Thu Jul 23 08:15:03 2026
Finding VCD file...
./dump.vcd
[2026-07-23 12:15:03 UTC] Opening EPwave...
```

Figure 2: Simulation log output from EDA Playground verifying the counter sequence.

4. Waveform Screenshot

The waveform below clearly demonstrates the 3-bit output ($q[2:0]$) transitioning through the required custom sequence, avoiding all unused states.

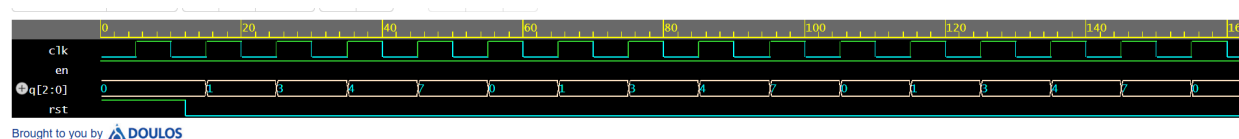


Figure 3: EPWave waveform confirming the accurate sequential state transitions.

5. Explanation

This task required designing a custom-sequence counter directly at the flip-flop level utilizing T flip-flops, representing the classical method for FSM design.

To determine the necessary logic gates for the flip-flop inputs, an excitation table must be derived. This table answers what input value a flip-flop needs to transition from its current state (Q) to its next state (Q_{next}). Because a T flip-flop operates on the principle of $Q_{next} = Q \oplus T$, the required T value is 1 if the state must toggle, and 0 if it must hold.

By plotting the specific sequence ($0 \rightarrow 1 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 0$) into the excitation table, the minimal Boolean equations for the three T flip-flops were derived:

- $t_2 = q_1 \cdot q_0$
- $t_1 = q_2 + q_0$
- $t_0 = q_2 + q_1 + \overline{q_0}$

In the SystemVerilog implementation, these equations are modeled using continuous `assign` statements. The T flip-flop behavior itself is modeled inside an `always_ff` procedural block using the bitwise XOR operator (`q <= q ^ t`), accurately capturing the toggle-on-1 logic. The testbench verifies that the design successfully executes 3 full cycles of the intended sequence without ever entering the undefined states (2, 5, or 6).